

Question 1: Conceptual/Entity-Relationship Model for an E-Commerce Database

I designed a conceptual/entity-relationship model capturing the mini-world description provided. It can be found in my GitHub, under folder 'Question 1: Conceptual/Entity-Relationship Modelling'.

(i) Use of Specialisation (if any) and whether they represent partial or total specialisations

There is one instance of specialisation in my E-R model, specifically total specialisation under the superclass 'PAYMENT METHOD', which contains attributes 'SerialCard Number' (primary key), and 'Expiry Date'. It is stated that "customers can register different payment methods, such as credit/debit card and voucher/gift card" (from the mini-world description), so I determined these to be the only payment options for simplicity. Consequently, I created these as disjoint subclasses; a payment method can only be one type, or the other (either a credit/debit card or a voucher/gift card).

(ii) Assumptions Related to Cardinality

Relationship I: CUSTOMER and PAYMENT METHOD through PAYS BY

- CUSTOMER to PAYMENT METHOD has (0, N) cardinality as a customer can have many payment methods, and optional (zero) minimum (no payment method registered).
- PAYMENT METHOD to CUSTOMER has (1, N) cardinality as some customers can have the same registered card, with each payment method belonging to at least one customer.

Relationship II: CUSTOMER and REVIEW through WRITES

- CUSTOMER to REVIEW has (0, N) cardinality as writing a review is optional, and if a customer chooses to write a review, they can write multiple reviews on any products ordered.
- REVIEW to CUSTOMER has (1, 1) cardinality as each review must be associated with (or written by) one customer, and only one customer

Relationship III: REVIEW and PRODUCT through RECEIVES

- REVIEW to PRODUCT has (1, 1) cardinality as when there is a review, it's on one product
- PRODUCT to REVIEW has (0, N) cardinality as each product does not necessarily have a review (as it is optional for customers), but each product can have multiple reviews about it

Relationship IV: PRODUCT and BASKET through ADDED TO

- PRODUCT to BASKET has (0, N) cardinality as a product does not have to be in a basket (it can exist independently), but products can be in multiple baskets as well.
- BASKET to PRODUCT has (0, N) cardinality as a basket can be empty (it doesn't necessarily contain products), or it can contain many products.

Relationship V: CUSTOMER and ORDER through PLACES AN

- CUSTOMER to ORDER has (0, N) cardinality as a customer could not have ordered anything (they could just have items in their basket, for example), or have multiple orders.
- ORDER to CUSTOMER has (1, 1) cardinality as each order belongs to one customer.

Relationship VI: CUSTOMER and BASKET through HAS A

- CUSTOMER to BASKET has (1, 1) cardinality as each customer has one assigned basket, and though it does not have to contain anything, this does not mean it does not exist.
- BASKET to CUSTOMER has (1, 1) cardinality as each basket belongs to one customer.

Relationship VII: ORDER and DELIVERY through DELIVERS

- ORDER to DELIVERY has (1, 1) cardinality as each instance of an order necessarily has one instance of delivery, there is no scenario where an order exists without a delivery instance.
- DELIVERY to ORDER has (1, 1) cardinality as there cannot be a delivery without an order

(NOTE: this relationship is simplified to ignore the real-world possibility of multiple part shipping)

RELATIONSHIP VIII: DELIVERY and RETURN through IS_RETURNED

- DELIVERY to RETURN has a cardinality of (0, 1) as a delivery can optionally be returned, and one return is assigned to one delivery (for simplicity, there are no single product returns)
- RETURN to DELIVERY has a cardinality of (1, 1) as if there is an instance of return, this belongs to one delivery instance (see earlier note about multiple part shipping)

(iii) Use of NULL Values (if an attribute is mandatory or not in a given entity)

There are some optional attributes in my model, but the majority of them are mandatory. Therefore, for report conciseness, the only optional attribute is 'Building_NameNumber' from CUSTOMER.

(iv) Other Aspects Adapted From the Provided Context.

I have adapted to my model from the provided context. These are detailed below.

- I created the 'Customer_ID' attribute as a shorter primary key for CUSTOMER for more efficient indexing, and searching opposed to email, which is still a unique identifier.
- I created the 'Is_Available' attribute for the PRODUCT entity, to simplify queries regarding a product's stock quantity (for example, if a customer was to order a product, it immediately identifies if that item was in stock, and then queries about stock quantity can continue).
- I am working under the assumption that if a customer chooses to order the products in their basket (held in the ADDED_TO entity), the details of the order, such as 'Product_Number', 'Desired_Quantity' only if 'Stock_Quantity' >= 'Desired_Quantity', are copied to ORDERED_PRODUCTS, and attributes calculated from these, such as 'Product_Subtotal', 'Grand_Total', etc. are calculated and stored in ORDER.
- It is stated that each review contains a product name, but does not specify an identifying attribute, so I have changed this so that review contains the attribute 'Product_Number'.
 - I did this as each product name is "short", and could be unidentifiable (for example, it states "office chair", and if there were more than one, from different brands, in different colours, dimensions, etc. it would be hard to differentiate reviews).

Question 2: Relational Model of E-Commerce Database Tables

I developed my conceptual/entity-relationship model into a relational schema. It can be found in my GitHub, under folder 'Question 2: Relational Modelling'.

(i) Adaptations from the Conceptual/E-R Model

As seen in Relationship IV (PRODUCT and BASKET through ADDED_TO), this is a many-to-many relationship, so it is its own table in the relational model. It stores products in customers' baskets.

(ii) Constraints Related to MIN, MAX Cardinalities, or Optional vs. Mandatory Participation

I have no comments to make on this that have not been made previously.

Question 3: Specificities of Database Creation

From the relational schema, I used ERDPlus functionality to generate SQL code. I altered it to add maximum characters (e.g. VARCHAR(12)). The code can be found in my GitHub, under 'schema.sql'.

(NOTE: ORDER changed to CUSTOMER ORDER, CREDIT/DEBIT to CREDIT_DEBIT and VOUCHER/GIFT CARD to VOUCHER_GIFTCARD)

(i) The Order of Table and View Creation

As the order of table creation is important due to foreign key constraints, I made sure to create the referenced table before the referencing table (e.g. **CUSTOMER** before **PAYMENT METHOD**).

- (1) CUSTOMER
- (2) PAYMENT METHOD
- (3) CREDIT_DEBIT
- (4) VOUCHER_GIFTCARD
- (5) PRODUCT
- (6) BASKET
- (7) ADDED_TO
- (8) ORDERED_PRODUCT
- (9) CUSTOMER_ORDER
- (10) DELIVERY
- (11) RETURN
- (12) REVIEW

(ii) Any Relaxation of Foreign Key Constraints

I did not need to relax any foreign key constraints to allow the database to be populated with data, as I ensured that the insertion of data into my database followed the order of table creation.

Question 4: Data Loading

All data loaded using Faker() can be found under 'data_generated.py', and INSERT INTO statements in 'manual_data_entry.sql'.

I analysed the provided Kaggle DataFrames, but I realised this would require a lot of data handling, and a mixture of importing data from these sources and using ChatGPT to help me generate Faker() data, so I decided to use the Faker() library to generate fake data, convert these into .csv files and import them into each table. I struggled to create fake reviews using Faker(), so I asked ChatGPT to generate INSERT INTO statements for this table, manually providing it with data to populate the database with five reviews as it was a tedious task and I had to detail customers and the products.

Question 5: SQL Programming

I have the required SQL code for Questions 5.1 to 5.4. The outputs are detailed below, and the code for each query is in my GitHub under folder 'Questions 5 & 6'.

Question 5.1: Customers and their Orders

I structured my database so that not all customers have ordered products. Therefore, I retrieved all customers' names and emails from the CUSTOMER table, and used LEFT JOIN with the CUSTOMER_ORDER table to retrieve the order numbers and (grand) total paid for those who have ordered. I used LEFT JOIN twice more to connect order numbers with all the products they have ordered (from the ORDERED_PRODUCTS table), and the names of these products (from the PRODUCTS table), allowing the result of this query to provide better insight into the products each customer has ordered. I aggregated this information using GROUP_CONCAT to reduce all product information for each order into two lists: Product_Numbers and Product_Names.

	Name	Email_Address	Order_Number	Date	Grand_Total	Product_Numbers	Product_Names
1	Valerie Mason	ashleypham@example.org	NULL	NULL	NULL	NULL	NULL
2	Lisa Daniel	qromero@example.org	NULL	NULL	NULL	NULL	NULL
3	Dylan Jackson	vbowers@example.net	69975	2023-11-28	83990.0	27, 46, 61, 66	Beverage, Cat Litter, Educational Toy, Bookcase
4	Emily Crosby	alison24@example.net	80165	2024-02-27	38808.0	2, 27, 44, 76	Pet Toy, Beverage, Drone, Dog Food
5	Emily Crosby	alison24@example.net	68435	2024-07-31	200.0	9	Smartwatch
6	Sara Ferguson	robertking@example.org	NULL	NULL	NULL	NULL	NULL
7	Donald Buchanan	cameronwilliams@example.org	78721	2024-06-16	16435.0	5, 26	Bakery Item, Orib
8	Paula Vance	alexander01@example.com	53521	2024-06-12	28365.0	48, 71, 82	Laptop, Pen, Jacket
9	Richard Murphy	qstanley@example.org	98012	2024-07-06	8000.0	17	Notebook
10	Carolyn Davis	haydenhernandez@example.org	NULL	NULL	NULL	NULL	NULL
11	Wendy Warren	hemith@example.com	87278	2024-07-07	17835.0	52	Stroller
12	Rhonda Beltran	mortoncourtney@example.com	46162	2024-04-07	66760.0	34, 39, 40, 78, 98	Pen, Spice, Snack, Wristwatch, Smartwatch
13	Christina Fox	travis13@example.net	NULL	NULL	NULL	NULL	NULL
14	Candice Davis	zriley@example.com	NULL	NULL	NULL	NULL	NULL
15	Lauren Perkins	cassidy38@example.org	22953	2024-06-11	26900.0	7, 66, 93	Office Chair, Bookcase, Beverage
16	Lauren Perkins	cassidy38@example.org	81269	2024-07-20	10048.0	71	Pen
17	Daniel Miller	catherinegreen@example.net	33138	2024-02-08	20385.0	13, 19, 41	Educational Toy, Supplement, Baby Clothing
18	Daniel Miller	catherinegreen@example.net	89673	2024-08-01	7595.0	80	Diaper
19	Tracy May	salinasmeredith@example.com	74288	2023-12-31	95505.0	10, 25, 46, 51, 77	Pen, Headphone, Laptop, Cat Litter, Spice
20	Tracy May	salinasmeredith@example.com	61731	2024-02-05	77945.0	20, 27, 39, 93	Earring, Beverage, Spice, Beverage

	Name	Email_Address	Order_Number	Date	Grand_Total	Product_Numbers	Product_Names
20	Tracy May	salinasmeredith@example.com	61731	2024-02-05	77945.0	20, 27, 39, 93	Earring, Beverage, Spice, Beverage
21	Tracy May	salinasmeredith@example.com	44801	2024-08-29	13983.75	11, 71, 74, 91	Pet Bed, Pen, Sofa, Diaper
22	Robert Miller	johnmatthews@example.net	82346	2024-02-29	4650.0	2	Pet Toy
23	Brittany Rocha	heathbrian@example.com	70074	2024-04-30	0.0	8, 59	Pen, Dog Food
24	Sarah Bauer	kristen91@example.com	71493	2024-01-06	27625.0	75	Smartphone
25	Sarah Bauer	kristen91@example.com	86246	2024-03-26	47990.0	8, 27, 30, 67, 72	Pen, Beverage, Bakery Item, Jean, Bracelet
26	Sarah Bauer	kristen91@example.com	77655	2024-05-26	20805.0	5, 47	Bakery Item, Hair Care
27	Kayla McKnight	dawnthompson@example.com	NULL	NULL	NULL	NULL	NULL
28	Mr. Charles Cooley	tare89@example.net	64722	2024-06-03	22750.0	72, 91	Bracelet, Diaper
29	Michele Williams	micheal29@example.com	58819	2024-06-13	71910.0	12, 16, 65, 73	Cat Litter, Bed, Accessory, Fragrance
30	Cory Hoffman	ericamiller@example.com	NULL	NULL	NULL	NULL	NULL
31	Samantha Short	patechelsea@example.org	96328	2024-09-14	54340.0	6, 7, 9, 23, 97	Skinicare Product, Office Chair, Smartwatch, Diaper, Shoe
32	Denise Davis	christopherwalker@example.net	NULL	NULL	NULL	NULL	NULL
33	Edward Modaniel	hellott@example.com	NULL	NULL	NULL	NULL	NULL
34	Whitney Ingram	medinaalec@example.net	20917	2024-04-25	44275.0	37, 93	Cat Litter, Beverage
35	Whitney Ingram	medinaalec@example.net	57180	2024-07-15	18835.0	34, 48, 92	Pen, Laptop, Fish Tank
36	Whitney Ingram	medinaalec@example.net	89617	2024-09-11	23180.0	30, 83, 89	Bakery Item, Office Chair, Beverage
37	James Shaw	jensenrobert@example.com	11480	2024-04-12	8295.0	15, 82, 97	Orib, Jacket, Shoe
38	James Shaw	jensenrobert@example.com	86309	2024-07-27	52000.0	25, 44, 83	Headphone, Drone, Office Chair
39	Adam Moses	samantha25@example.org	NULL	NULL	NULL	NULL	NULL

	Name	Email_Address	Order_Number	Date	Grand_Total	Product_Numbers	Product_Names
39	Adam Moses	samantha25@example.org	NULL	NULL	NULL	NULL	NULL
40	Joe Dawson	steven62@example.net	NULL	NULL	NULL	NULL	NULL
41	Jeffrey Willis	garretthris@example.net	NULL	NULL	NULL	NULL	NULL
42	Sandra Lopez	williamsanders@example.net	89684	2024-01-28	126660.0	65, 77, 85, 95, 99	Accessory, Spice, Dining Table, Accessory, Accessory
43	Sandra Lopez	williamsanders@example.net	93146	2024-04-12	27840.0	70	Stroller
44	Lawrence Gibbs	cattlinlee@example.net	31373	2024-07-13	3955.0	9, 38	Smartwatch, Smartphone
45	Lawrence Gibbs	cattlinlee@example.net	74185	2024-10-28	75190.0	38, 75, 78	Smartphone, Smartphone, Wristwatch
46	Angela Erickson	robert80@example.net	NULL	NULL	NULL	NULL	NULL
47	Richard Ward	robert24@example.net	54450	2023-12-07	73085.0	2, 3, 28	Pet Toy, Bakery Item, Planner
48	Richard Ward	robert24@example.net	10446	2024-04-14	20290.0	28, 42	Planner, Pet Toy
49	Laura Harvey	nrogers@example.net	NULL	NULL	NULL	NULL	NULL
50	Suzanne Scott	lwilcox@example.com	61943	2023-12-15	28710.0	52	Stroller
51	Jessica Peterson PhD	torreaalexis@example.com	65868	2024-07-30	83015.0	3, 4, 61, 74, 85	Bakery Item, Snack, Educational Toy, Sofa, Dining Table
52	Danielle Hall	mstewart@example.org	94323	2024-05-02	17185.0	14, 30, 56	Beverage, Bakery Item, Snack
53	Danielle Hall	mstewart@example.org	57314	2024-05-27	49680.0	4, 5, 51, 75	Snack, Bakery Item, Cat Litter, Smartphone
54	Misty Hubbard	peterbrown@example.net	NULL	NULL	NULL	NULL	NULL
55	John Jones	michael79@example.com	78926	2024-06-05	12190.0	24, 30, 75, 76	Educational Toy, Bakery Item, Smartphone, Dog Food
56	John Jones	michael79@example.com	28983	2024-11-09	27805.0	33, 46, 56, 83	Organic Food, Cat Litter, Snack, Office Chair
57	Katrina Preston	andreamercado@example.org	49491	2024-10-29	46265.0	3, 35, 61, 71	Bakery Item, Organic Food, Educational Toy, Pen
58	John Dixon	chuffman@example.net	NULL	NULL	NULL	NULL	NULL

	Name	Email_Address	Order_Number	Date	Grand_Total	Product_Numbers	Product_Names
58	John Dixon	chuffman@example.net	NULL	NULL	NULL	NULL	NULL
59	Tricia Sanchez	cassie91@example.com	NULL	NULL	NULL	NULL	NULL
60	Lisa Wallace	christianjones@example.net	NULL	NULL	NULL	NULL	NULL
61	Cory Cantrell	williamhuerta@example.com	80273	2024-01-12	22325.0	4, 56, 64, 87	Snack, Snack, Hair Care, Wristwatch
62	Jessica Chavez	sfreeman@example.com	66488	2024-05-13	9645.0	2, 82	Pet Toy, Jacket
63	Olivia Murphy	ann46@example.org	51590	2024-05-19	45680.0	72, 77	Bracelet, Spice
64	Joshua Brandt	coopermichael@example.net	76083	2024-01-06	62790.0	10, 39, 49, 68, 83	Pen, Spice, Office Chair, Pet Bed, Office Chair
65	Brian Pratt	davidrichards@example.org	NULL	NULL	NULL	NULL	NULL

Question 5.2: All Customer Basket Details and Gift Card Balances for Birthday Offers

I structured my database so that not all customers have registered payment methods, and that a payment method is either a CREDIT_DEBIT or VOUCHER_GIFTCARD. Additionally, I made PAYMENT_METHOD a super-entity, with VOUCHER_GIFTCARD being a sub-entity, so after selecting customer IDs, names, email addresses, and dates of birth from customers who have items in their basket (using INNER JOIN), I can reference this table through using LEFT JOIN on PAYMENT_METHOD and VOUCHER_GIFTCARD to retrieve the total and current balances of any vouchers/gift cards (if any, as if they do not have a registered voucher/gift card these values are null). Furthermore, I used GROUP_CONCAT again to generate a list of products in their baskets.

	Customer_ID	Name	Email_Address	Date_of_Birth	Current_Balance	Product_Numbers
1	3	Dylan Jackson	vbowers@example.net	1926-06-17	NULL	1, 5, 6, 7, 8, 12, 16, 19, 23, 25
2	4	Emily Crosby	alison24@example.net	2005-09-18	18.0	2, 3, 5, 8, 15, 16, 19, 21, 22, 23
3	9	Carolyn Davis	haydenhernandez@example.org	1952-05-22	26.0	12, 14, 20, 22
4	13	Candice Davis	zriley@example.com	1993-06-21	21.0	10, 15, 23, 24
5	14	Lauren Perkins	cassidy38@example.org	1941-02-25	15.0	5, 6, 9, 10, 11, 16, 19, 21, 23, 25
6	16	Tracy May	salinasmeredith@example.com	1981-01-05	NULL	1, 2, 6, 12, 13, 15, 18, 19, 22, 23
7	18	Brittany Rocha	heathbrian@example.com	1951-06-30	36.0	1, 6, 7, 11, 19, 20, 21, 23, 24, 25
8	20	Kayla McKnight	dawnthompson@example.com	1911-11-04	NULL	2, 5, 6, 7, 11, 12, 15, 22, 24, 25
9	21	Mr. Charles Cooley	tara89@example.net	1947-04-26	NULL	4, 5, 6, 7, 9, 11, 13, 15, 17, 25
10	22	Michele Williams	micheal29@example.com	1965-01-23	NULL	9, 11, 13, 18
11	24	Samantha Short	patelchelsea@example.org	1965-03-09	NULL	1, 2, 3, 8, 11, 15, 18, 20, 22, 24
12	25	Denise Davis	christopherwalker@example.net	1994-01-07	NULL	3, 5, 6, 8, 10, 14, 15, 17, 19, 20
13	26	Edward McDaniel	ielliott@example.com	1949-11-05	9.0	6, 12, 13, 15, 17, 18, 19, 20, 21, 23
14	27	Whitney Ingram	medinaalec@example.net	1926-11-29	11.0	6, 10, 11, 25
15	31	Jeffrey Willis	garrettchris@example.net	1916-09-11	39.0	6, 7, 8, 18
16	33	Lawrence Gibbs	caitlinlee@example.net	1961-03-28	25.0	2, 11, 20, 25
17	34	Angela Erickson	robert80@example.net	1988-01-16	NULL	11, 15, 18, 19
18	37	Suzanne Scott	lwilcox@example.com	1954-12-28	NULL	6, 10, 14, 15, 17, 18, 20, 21, 22, 24
19	39	Danielle Hall	mstewart@example.org	1988-04-18	0.0	4, 5, 14, 23
20	40	Misty Hubbard	peterbrown@example.net	2004-03-18	NULL	3, 10, 11, 16, 17, 19, 21, 22, 23, 25
21	43	John Dixon	chuffman@example.net	1978-07-08	17.0	2, 6, 7, 13, 15, 16, 17, 19, 22, 23
22	44	Tricia Sanchez	cassie91@example.com	1954-11-13	NULL	4, 5, 6, 11, 13, 14, 16, 17, 18, 24
23	46	Cory Cantrell	williamhuerta@example.com	1925-08-27	3.0	1, 6, 8, 9, 10, 11, 14, 16, 21, 23
24	47	Jessica Chavez	sfreeman@example.com	1932-06-25	NULL	16, 18, 21, 22
25	48	Olivia Murphy	ann46@example.org	1961-08-11	12.0	4, 5, 15, 18

Question 5.3: Top Two Best-Selling Products by Category and Their Total Sales

I structured this query first with a WITH construct to define a RankedProducts table that will only be used in this particular query (similar to a “temporary view”), allowing me to immediately access and query the ranked products. I used the window function, ROW_NUMBER so that each CATEGORY ranks each product within the category under ‘Total_Sales’ (the number of products sold multiplied by the price of the product), and then filtered RankedProducts to show the first two rows for each category. The results are ordered by category for clarity in reading the results.

I used the WITH construct instead of CREATE VIEW as it avoided creating unnecessary, persistent objects in the database when I only need the result temporarily.

	Category	Product_Name	Product_Number	Total_Sales	RowNum
1	Baby Products	Stroller	52	46545.0	1
2	Baby Products	Stroller	70	27840.0	2
3	Books and Stationery	Pen	71	35525.0	1
4	Books and Stationery	Planner	28	32000.0	2
5	Clothing and Apparel	Accessory	65	66960.0	1
6	Clothing and Apparel	Accessory	99	20300.0	2
7	Electronics	Smartphone	75	96900.0	1
8	Electronics	Headphone	25	83660.0	2
9	Furniture	Dining Table	85	71440.0	1
10	Furniture	Office Chair	83	30900.0	2
11	Grocery and Gourmet Food	Bakery Item	3	93575.0	1
12	Grocery and Gourmet Food	Spice	77	91180.0	2
13	Health and Beauty	Skincare Product	6	31995.0	1
14	Health and Beauty	Hair Care	47	16800.0	2
15	Jewelry and Watches	Wristwatch	78	39100.0	1
16	Jewelry and Watches	Earring	20	33015.0	2
17	Pet Supplies	Cat Litter	51	47565.0	1
18	Pet Supplies	Cat Litter	37	35145.0	2
19	Toys and Games	Educational Toy	61	4850.0	1
20	Toys and Games	Educational Toy	13	3520.0	2

Question 5.4: Month-over-Month Sales Growth with Total Sales and Refund Adjustments

As I attempted to generate this query, I realised the best way was to create a view, called MonthlySales. It had the columns ‘Year’, ‘Month’, and ‘Total_Sales’ (calculated using the SUM(CASE()) functions). This calculation looked for if the order had been delivered, is pending or is postponed (discounting where the delivery has been cancelled). It also used the RETURN table to make sure that if an order was returned successfully, this did not count toward ‘Total_Sales’. The GROUP BY and ORDER BY functions meant that this data could be easily read, and using LEFT JOIN ensured items without deliveries/returns were still included to gain a holistic view of sales data.

(NOTE: The formula for percentage change is $\frac{(New - Original)}{Original} * 100$)

The creation of this view meant that gaining the final piece of required information (‘Percentage_Change’) was easier. I used the LAG window function, as it can compare the current row to the previous one. As the formula for percentage change includes division, there is a “!= 0” which if FALSE (i.e. the previous value is 0), it returns 0 to avoid zero division error. It then calculates this value for each row, resulting in the following.

However, gaining the result below required some manual changing of the database, as there was one inconsistency – a delivery with status ‘pending’ had a ‘completed’ return status. As there were few returns, I easily fixed this, and the query ran smoothly.

	Year	Month	Total_Sales	Percentage_Change
1	2023	11	0	0
2	2023	12	197300.0	0
3	2024	01	189450.0	-3.97871262037506
4	2024	02	38805.0	-79.5170229612035
5	2024	03	47990.0	23.6696302022935
6	2024	04	75740.0	57.8245467806793
7	2024	05	133305.0	76.0034327964088
8	2024	06	28365.0	-78.721728367278
9	2024	07	40835.0	43.9626300017627
10	2024	08	94660.0	131.81094649198
11	2024	09	77520.0	-18.1069089372491
12	2024	10	0	-100.0
13	2024	11	0	0

Question 6: Database Consistency

I wrote a trigger, used to maintain database consistency. The situation I chose and the outputs are detailed below, and the code for my trigger is in my GitHub under folder 'Questions 5 & 6'

Trigger (iii) Customer Tries to Open a New Return When They Already Have a Pending Return

The trigger may be called prior to INSERT on the RETURN table, to avoid two attempts at a return. It uses EXISTS to check if there is already a value in the RETURN table for the specific order. JOIN is used on CUSTOMER_ORDER and RETURN to check if the customer making the new return is the same one who made the original one, and then the code filters if there is a current return that is pending or has been denied. If a RETURN exists, an error is raised, if not, the values are inserted.

```
Execution finished without errors.
Result: query executed successfully. Took 0ms
At line 1:
CREATE TRIGGER MultipleReturnRequests
BEFORE INSERT ON RETURN
FOR EACH ROW
BEGIN
  SELECT CASE
    WHEN EXISTS (
      SELECT 1
      FROM CUSTOMER_ORDER AS co JOIN RETURN AS r ON co.Order_Number = r.Order_Number
      WHERE co.Customer_ID = (SELECT Customer_ID FROM CUSTOMER_ORDER WHERE Order_Number = NEW.Order_Number)
      AND r.Status = 'pending' OR r.Status = 'denied'
    ) LIMIT 1
  )
  THEN RAISE (ABORT, 'ERROR: customer already has a pending return for this order.')
END;
END;
```

(i) A Query That Passes the Trigger (i.e., normal situation, no errors)

```
Execution finished without errors.
Result: query executed successfully. Took 1ms
At line 1:
-- CREATE TRIGGER MultipleReturnRequests
-- BEFORE INSERT ON RETURN
-- FOR EACH ROW
-- BEGIN
--   SELECT CASE
--     WHEN EXISTS (
--       SELECT 1
--       FROM CUSTOMER_ORDER AS co JOIN RETURN AS r ON co.Order_Number = r.Order_Number
--       WHERE co.Customer_ID = (SELECT Customer_ID FROM CUSTOMER_ORDER WHERE Order_Number = NEW.Order_Number)
--       AND r.Status = 'pending' OR r.Status = 'denied'
--     ) LIMIT 1
--   )
--   THEN RAISE (ABORT, 'ERROR: customer already has a pending return for this order.')
-- END;
-- END;
--
-- INSERT INTO RETURN (Ticket_Number, Order_Number, Start_Date, Due_Date, Refund_Total, Status)
```

(ii) A Query That Violates Any Condition and “Fires” the Trigger (i.e. generates error message)

```
Execution finished with errors.
Result: ERROR: customer already has a pending or denied return for this order.
At line 1:
-- CREATE TRIGGER MultipleReturnRequests
-- BEFORE INSERT ON RETURN
-- FOR EACH ROW
-- BEGIN
--   SELECT CASE
--     WHEN EXISTS (
--       SELECT 1
--       FROM RETURN AS r JOIN CUSTOMER_ORDER AS co ON co.Order_Number = r.Order_Number
--       WHERE co.Customer_ID = (SELECT Customer_ID FROM CUSTOMER_ORDER WHERE Order_Number = NEW.Order_Number)
--       AND r.Status IN ('pending', 'denied') -- Correct condition
--       LIMIT 1
--     )
--   THEN RAISE (ABORT, 'ERROR: customer already has a pending or denied return for this order.')
--   END;
-- END;
```

A Statement on the Use of Generative AI Tools, As Per School and Course-Specific Policies

In response to the guidance on the full authorised use of generative AI in assessment, I have utilised generative AI tools throughout the learning and assessment process as permitted. These tools were particularly useful for generating synthetic data for my database application. However, I made sure to use these tools responsibly to ensure my learning and development align with course expectations.

For Questions 4, 5, and 6, I mistakenly deleted ChatGPT logs that I queried for help. To avoid any undue repercussions, I have stated in my SQL code (inside my GitHub repository) the sections of this assignment that were aided by ChatGPT, including the entirety of my synthetic data generation. This is to ensure transparency and adherence to the guidelines while mitigating any concerns.